Bhawesh War

Parallel & Distributed Computing - UCS645

Lab Assignment 1 - Report

Q1: DAXPY Loop using OpenMP

Objective

The objective of this experiment is to study the performance of the DAXPY operation using both serial and parallel implementations. The DAXPY operation is a fundamental vector computation commonly used in scientific and numerical applications. It is defined as:

$$X[i] = a \times X[i] + Y[i]$$

where a is a scalar constant and X and Y are one-dimensional vectors of size N. The goal is to analyze how the execution time changes with an increase in the number of threads and to observe the speedup obtained using OpenMP.

1. Algorithm / Pseudocode

Serial Algorithm:

```
Initialize arrays X and Y
Start timer
For i = 0 to N-1:
    X[i] = a * X[i] + Y[i]
Stop timer
Print execution time
```

Parallel Algorithm (OpenMP):

```
Initialize arrays X and Y
Start timer
#pragma omp parallel for
For i = 0 to N-1:
    X[i] = a * X[i] + Y[i]
Stop timer
Print execution time
```

2. Implementation Details

The program is implemented in C using the OpenMP API for parallelization. The code was compiled using the GNU Compiler Collection (gcc) with the `-fopenmp` flag on Ubuntu running through Windows Subsystem for Linux (WSL). The execution time was measured using the OpenMP function `omp_get_wtime()`.

3. Results

The experimental results are divided into two parts: Serial execution and Parallel execution.

Serial Execution

The serial version of the DAXPY program was executed using a single thread. The execution time obtained is shown below:

$$\textbf{Execution Time (Serial)} = 0.000360 \text{ seconds}$$

This value is taken as the baseline for calculating the speedup of the parallel versions.

Parallel Execution

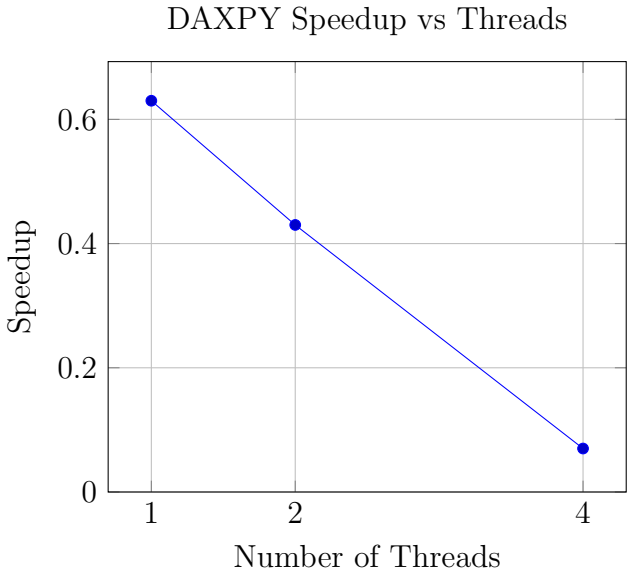
The parallel version of the DAXPY program was executed using different numbers of threads. The execution times and corresponding speedup values are shown in the table below.

Threads	Time (seconds)	Speedup
1	0.000572	0.63
2	0.000843	0.43
4	0.005162	0.07

Table 1: Parallel Execution Time and Speedup for DAXPY

4. Graph

The following graph shows the variation of speedup with respect to the number of threads for the parallel implementation.



5. Inference / Conclusion

From the experimental results, it is observed that the parallel implementation performs worse than the serial implementation for this particular problem size. This is because the DAXPY operation involves very small computation per iteration, and the overhead of thread creation, synchronization, and scheduling dominates the actual computation time.

As the number of threads increases, the execution time further increases due to excessive overhead and limited computational workload. This demonstrates that parallelization is not always beneficial for small problem sizes and that performance gain depends heavily on the computation-to-overhead ratio.

Hence, this experiment highlights an important concept in parallel computing: parallel execution is effective only when the problem size is sufficiently large to amortize the overhead introduced by multithreading.

Q2: Matrix Multiplication using OpenMP

Objective

The objective of this experiment is to implement matrix multiplication in both serial and parallel forms and analyze the performance improvement obtained through parallelization. The experiment also compares two different parallel approaches: 1D threading and 2D threading.

Given two square matrices A and B of size $N \times N$, the resulting matrix C is computed as:

$$C[i][j] = \sum_{k=0}^{N-1} A[i][k] \times B[k][j]$$

1. Algorithm / Pseudocode

Serial Algorithm:

```
For i = 0 to N-1:
    For j = 0 to N-1:
        C[i][j] = 0
        For k = 0 to N-1:
            C[i][j] += A[i][k] * B[k][j]
```

Parallel Algorithm (1D Threading):

```
#pragma omp parallel for
For i = 0 to N-1:
    For j = 0 to N-1:
        For k = 0 to N-1:
            C[i][j] += A[i][k] * B[k][j]
```

Parallel Algorithm (2D Threading):

```
#pragma omp parallel for collapse(2)
For i = 0 to N-1:
    For j = 0 to N-1:
        For k = 0 to N-1:
            C[i][j] += A[i][k] * B[k][j]
```

2. Implementation Details

The program is implemented in C using OpenMP. The serial and parallel versions were compiled using gcc with the `-fopenmp` flag. Execution time was measured using the OpenMP timing function `omp_get_wtime()` on Ubuntu (WSL).

3. Results

The experimental results are divided into three parts: Serial execution, Parallel 1D execution, and Parallel 2D execution.

Serial Execution

The serial matrix multiplication program was executed using a single thread.

$$\textbf{Execution Time (Serial)} = 3.265237 \text{ seconds}$$

This value is used as the baseline for speedup calculation.

Parallel Execution (1D Threading)

Threads	Time (seconds)	Speedup
1	2.085524	1.57
2	0.925925	3.53
4	0.857077	3.81
12	0.892837	3.66

Table 2: Parallel 1D Execution Results

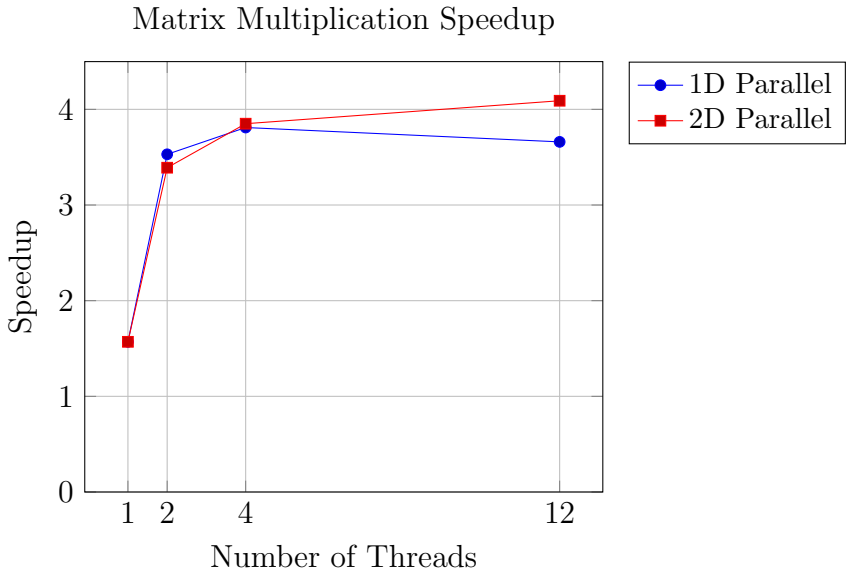
Parallel Execution (2D Threading)

Threads	Time (seconds)	Speedup
1	2.083497	1.57
2	0.963458	3.39
4	0.848223	3.85
12	0.797442	4.09

Table 3: Parallel 2D Execution Results

4. Graph

The following graph shows the variation of speedup with respect to the number of threads for both 1D and 2D parallel implementations.



5. Inference / Conclusion

From the results, it is observed that both parallel implementations significantly outperform the serial version. The execution time decreases as the number of threads increases up to a certain point.

The 2D parallel implementation performs better than the 1D version, especially at higher thread counts. This is because 2D threading distributes the workload more evenly among threads by parallelizing both row and column loops, leading to better load balancing and improved utilization of CPU cores.

However, after a certain number of threads, the performance gain starts to saturate due to factors such as memory bandwidth limitations and thread management overhead. This demonstrates that while parallelization improves performance, the speedup is bounded by hardware and system constraints.

Q3: Parallel Computation of π using OpenMP

Objective

The objective of this experiment is to compute an approximate value of π using numerical integration and analyze the performance improvement obtained through parallel execution. The experiment demonstrates how multiple threads can cooperate to compute a single numerical result.

The value of π is calculated using the integral:

$$\pi = \int_0^1 \frac{4}{1+x^2}\,dx$$

1. Algorithm / Pseudocode

Serial Algorithm:

```
Initialize sum = 0
For i = 0 to N-1:
    x = (i + 0.5) / N
    sum += 4 / (1 + x*x)
pi = sum / N
```

Parallel Algorithm:

```
Initialize sum = 0
#pragma omp parallel for reduction(+:sum)
For i = 0 to N-1:
    x = (i + 0.5) / N
    sum += 4 / (1 + x*x)
pi = sum / N
```

2. Implementation Details

The program is implemented in C using OpenMP. The reduction clause is used to avoid race conditions while accumulating the partial sums computed by different threads. The program is compiled using gcc with the `-fopenmp` flag and executed on Ubuntu (WSL).

3. Results

The experimental results are divided into Serial execution and Parallel execution.

Serial Execution

The serial version of the program was executed using a single thread.

Execution Time (Serial) = 0.769979 seconds

This value is used as the baseline for speedup calculation.

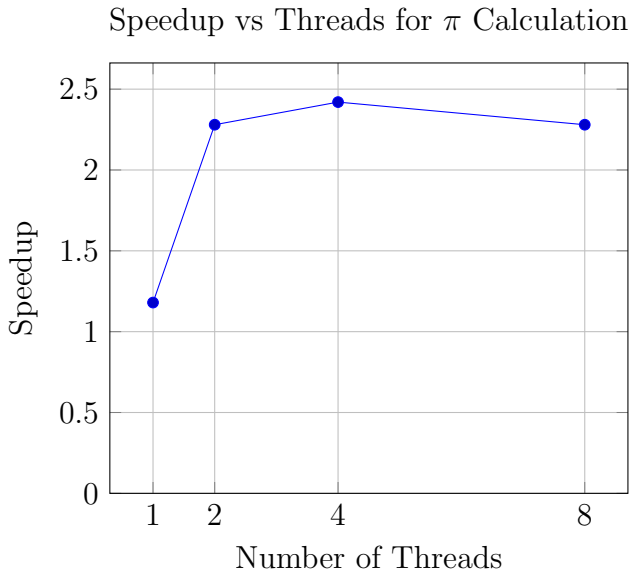
Parallel Execution

Threads	Time (seconds)	Speedup
1	0.653224	1.18
2	0.338032	2.28
4	0.318025	2.42
8	0.337795	2.28

Table 4: Parallel Execution Time and Speedup for π Calculation

4. Graph

The following graph shows the variation of speedup with respect to the number of threads.



5. Inference / Conclusion

From the experimental results, it is observed that parallel execution significantly reduces the computation time for calculating the value of π. The speedup increases as the number of threads increases up to 4 threads, after which the performance saturates.

The slight decrease in speedup at 8 threads can be attributed to thread management overhead and increased synchronization cost. This demonstrates that parallel performance is limited by system resources and that maximum speedup is achieved when the number of threads is close to the number of available CPU cores.

This experiment highlights the effectiveness of OpenMP reduction in parallel numerical computations and illustrates the concept of diminishing returns in parallel systems.